Multi-Dimensional Arrays

CIS111 – Week 8 lecture

Sait Erkovan

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-29

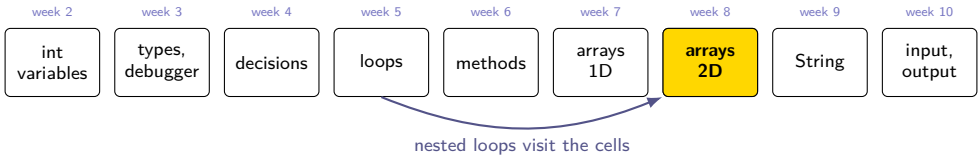
Outline

1. Why tables?
2. Creating and using a table
3. Common table algorithms
4. Tables and methods
5. More dimensions
6. Summary

Why tables?

Week 8: from a list to a table

Where we are



You know (week)	Today
an array `int[] a`, an element `a[i]` (7)	a table `int[][] t`, a cell `t[r][c]`
`a.length` (7)	`t.length` rows, `t[r].length` columns
one loop visits an array (7)	two nested loops visit a table (5)
sum, best so far, search, count (7)	per row, per column, or over all cells
a method with an `int[]` parameter (7)	a method with an `int[][]` parameter

Today's problem: three lists, or one table?

Quiz scores of 3 students in 4 quizzes: what is the total of each student?

codeoutput/ThreeLists.java — Java

```
3         int[] s0 = {72, 85, 90, 64};
4         int[] s1 = {55, 61, 70, 80};
5         int[] s2 = {90, 88, 93, 99};
6         int t0 = 0, t1 = 0, t2 = 0;
7         for (int q = 0; q < 4; q++) {
8             t0 = t0 + s0[q];
9             t1 = t1 + s1[q];
10            t2 = t2 + s2[q];
11        }
12        System.out.println(t0);
13        System.out.println(t1);
14        System.out.println(t2);
```

Today's problem: three lists, or one table?

Quiz scores of 3 students in 4 quizzes: what is the total of each student?

codeoutput/ThreeLists.java — Java

```
3         int[] s0 = {72, 85, 90, 64};
4         int[] s1 = {55, 61, 70, 80};
5         int[] s2 = {90, 88, 93, 99};
6         int t0 = 0, t1 = 0, t2 = 0;
7         for (int q = 0; q < 4; q++) {
8             t0 = t0 + s0[q];
9             t1 = t1 + s1[q];
10            t2 = t2 + s2[q];
11        }
12        System.out.println(t0);
13        System.out.println(t1);
14        System.out.println(t2);
```

Output — codeoutput/ThreeLists.txt

```
311
266
370
```

Q. And for 30 students?

Today's problem: three lists, or one table?

Quiz scores of 3 students in 4 quizzes: what is the total of each student?

codeoutput/ThreeLists.java — Java

```
3         int[] s0 = {72, 85, 90, 64};
4         int[] s1 = {55, 61, 70, 80};
5         int[] s2 = {90, 88, 93, 99};
6         int t0 = 0, t1 = 0, t2 = 0;
7         for (int q = 0; q < 4; q++) {
8             t0 = t0 + s0[q];
9             t1 = t1 + s1[q];
10            t2 = t2 + s2[q];
11        }
12        System.out.println(t0);
13        System.out.println(t1);
14        System.out.println(t2);
```

Output — codeoutput/ThreeLists.txt

```
311
266
370
```

Q. And for 30 students?

codeoutput/OneTable.java — Java

```
3         int[][] s = {{72, 85, 90, 64},
4                      {55, 61, 70, 80},
5                      {90, 88, 93, 99}};
6         for (int r = 0; r < s.length; r++) {
7             int total = 0;
8             for (int q = 0; q < s[r].length; q++) {
9                 total = total + s[r][q];
10            }
11            System.out.println(total);
12        }
```

Today's problem: three lists, or one table?

Quiz scores of 3 students in 4 quizzes: what is the total of each student?

codeoutput/ThreeLists.java — Java

```
3         int[] s0 = {72, 85, 90, 64};
4         int[] s1 = {55, 61, 70, 80};
5         int[] s2 = {90, 88, 93, 99};
6         int t0 = 0, t1 = 0, t2 = 0;
7         for (int q = 0; q < 4; q++) {
8             t0 = t0 + s0[q];
9             t1 = t1 + s1[q];
10            t2 = t2 + s2[q];
11        }
12        System.out.println(t0);
13        System.out.println(t1);
14        System.out.println(t2);
```

Output — codeoutput/ThreeLists.txt

```
311
266
370
```

codeoutput/OneTable.java — Java

```
3         int[][] s = {{72, 85, 90, 64},
4                      {55, 61, 70, 80},
5                      {90, 88, 93, 99}};
6         for (int r = 0; r < s.length; r++) {
7             int total = 0;
8             for (int q = 0; q < s[r].length; q++) {
9                 total = total + s[r][q];
10            }
11            System.out.println(total);
12        }
```

Output — codeoutput/OneTable.txt

```
311
266
370
```

Q. And for 30 students? Left: 30 arrays and 30 totals. Right: one name `s`; only the rows in lines 3–5 grow.

One name for a table: the parts

```
int[][] s = {{72, 85, 90, 64},      // row 0: student 0, quizzes 0 to 3
             {55, 61, 70, 80},      // row 1: student 1
             {90, 88, 93, 99}};     // row 2: student 2
System.out.println(s[1][2]);        // row 1, column 2
```

s

	0	1	2	3
0	72	85	90	64
1	55	61	70	80
2	90	88	93	99

row: `s[1]` is row 1, an `int[]`
column: `s[0][2]`, `s[1][2]`, `s[2][2]` form column 2
cell: `s[1][2]` is 70; row first, then column
size: `s.length` is 3 rows, `s[r].length` is 4 columns

$3 \times 4 = 12$ cells under one name. The rows are the students, the columns the quizzes.

Creating and using a table

Two indices, two loops

Declaring and creating: two steps

```
int[][] scores;              // 1. a variable that can refer to a table of ints
scores = new int[3][4];      // 2. the table itself: 3 rows, 4 columns, all 0

final int WEEKS = 4, DAYS = 7;
double[][] highs = new double[WEEKS][DAYS];   // both steps in one line
```

`double[][]`	`highs`	=	`new double[WEEKS][DAYS]`
element type and `[][]`	name		creates 4 rows of 7 elements; the rows come first

- Two pairs of brackets: `double[][]` is read "double array array", an array whose elements are rows
- Always `[rows][columns]`; named constants say which number is which

A first table

codeoutput/FirstTable.java — Java

```java
public class FirstTable {
    public static void main(String[] args) {
        int[][] t = new int[2][3];  // 2 rows, 3 columns, all 0
        t[0][0] = 5;
        t[0][2] = 7;
        t[1][1] = 9;                        // t[1][0] stays 0
        for (int r = 0; r < t.length; r++) {
            for (int c = 0; c < t[r].length; c++) {
                System.out.print(t[r][c] + " ");
            }
            System.out.println();
        }
        int[][] m = {{1, 2, 3}, {4, 5, 6}};  // rows in braces
        System.out.println(m.length + " x " + m[0].length);
        System.out.println(m[1][2]);
    }
}
```

Q. What does it print? Look at line 6.

A first table

codeoutput/FirstTable.java — Java

```
1  public class FirstTable {
2      public static void main(String[] args) {
3          int[][] t = new int[2][3];  // 2 rows, 3 columns, all 0
4          t[0][0] = 5;
5          t[0][2] = 7;
6          t[1][1] = 9;                          // t[1][0] stays 0
7          for (int r = 0; r < t.length; r++) {
8              for (int c = 0; c < t[r].length; c++) {
9                  System.out.print(t[r][c] + " ");
10             }
11             System.out.println();
12         }
13         int[][] m = {{1, 2, 3}, {4, 5, 6}};  // rows in braces
14         System.out.println(m.length + " x " + m[0].length);
15         System.out.println(m[1][2]);
16     }
17 }
```

Output
codeoutput/FirstTable.txt

```
5 0 7
0 9 0
2 x 3
6
```

Q. What does it print? Look at line 6. Nobody wrote `t[1][0]`: it keeps its start value 0.

A list of rows `{{...}, {...}}` gives the values: `m.length` rows of `m[0].length` cells.

Quick check: create

```
int[][] g = new int[2][5];
```

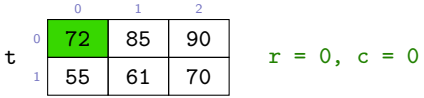
1. What is `g.length`?
2. What is `g[0].length`?
3. Which is the last cell?
4. What is `g[1][4]` right now?
5. Make the table `{{1, 2}, {3, 4}}` in one line.

Quick check: create

```
int[][] g = new int[2][5];
```

1. What is `g.length`? 2, the rows
2. What is `g[0].length`? 5, the cells of row 0
3. Which is the last cell? `g[1][4]`
4. What is `g[1][4]` right now? 0, the start value
5. Make the table `{{1, 2}, {3, 4}}` in one line. `int[][] h = {{1, 2}, {3, 4}};`

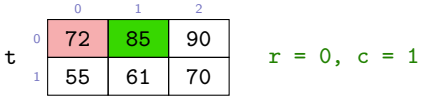
Visiting every cell: row by row




```
for (int r = 0; r < t.length; r++) {              // every row
    for (int c = 0; c < t[r].length; c++) {       // every column of row r
        System.out.print(t[r][c] + " ");
    }
    System.out.println();                         // row r is done
}
```

`t.length`: the number of rows. `t[r].length`: the length of row `r`. The inner loop runs completely for every `r`: the nested loops of week 5.

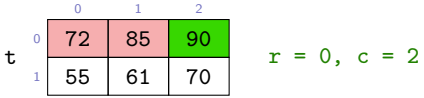
Visiting every cell: row by row




```
for (int r = 0; r < t.length; r++) {              // every row
    for (int c = 0; c < t[r].length; c++) {       // every column of row r
        System.out.print(t[r][c] + " ");
    }
    System.out.println();                         // row r is done
}
```

`t.length`: the number of rows. `t[r].length`: the length of row `r`. The inner loop runs completely for every `r`: the nested loops of week 5.

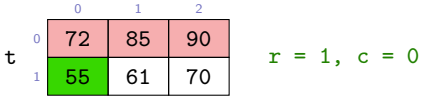
Visiting every cell: row by row




```
for (int r = 0; r < t.length; r++) {              // every row
    for (int c = 0; c < t[r].length; c++) {       // every column of row r
        System.out.print(t[r][c] + " ");
    }
    System.out.println();                         // row r is done
}
```

`t.length`: the number of rows. `t[r].length`: the length of row `r`. The inner loop runs completely for every `r`: the nested loops of week 5.

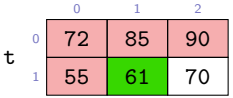
Visiting every cell: row by row




```
for (int r = 0; r < t.length; r++) {              // every row
    for (int c = 0; c < t[r].length; c++) {       // every column of row r
        System.out.print(t[r][c] + " ");
    }
    System.out.println();                         // row r is done
}
```

`t.length`: the number of rows. `t[r].length`: the length of row `r`. The inner loop runs completely for every `r`: the nested loops of week 5.

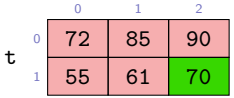
Visiting every cell: row by row




```
for (int r = 0; r < t.length; r++) {              // every row
    for (int c = 0; c < t[r].length; c++) {       // every column of row r
        System.out.print(t[r][c] + " ");
    }
    System.out.println();                         // row r is done
}
```

`t.length`: the number of rows. `t[r].length`: the length of row `r`. The inner loop runs completely for every `r`: the nested loops of week 5.

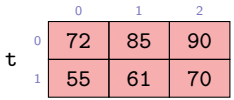
Visiting every cell: row by row




```
for (int r = 0; r < t.length; r++) {              // every row
    for (int c = 0; c < t[r].length; c++) {       // every column of row r
        System.out.print(t[r][c] + " ");
    }
    System.out.println();                         // row r is done
}
```

`t.length`: the number of rows. `t[r].length`: the length of row `r`. The inner loop runs completely for every `r`: the nested loops of week 5.

Visiting every cell: row by row



`r == t.length:  stop`

```
for (int r = 0; r < t.length; r++) {              // every row
    for (int c = 0; c < t[r].length; c++) {       // every column of row r
        System.out.print(t[r][c] + " ");
    }
    System.out.println();                         // row r is done
}
```

`t.length`: the number of rows. `t[r].length`: the length of row `r`. The inner loop runs completely for every `r`: the nested loops of week 5.

The enhanced for loop on a table

```
for (int[] row : t) { for (int x : row) { ... } }   // for each row, each x in it
```

The outer variable is a whole row, an `int[]`; the inner loop is the enhanced for of week 7 on that row.

codeoutput/ForEachTable.java — Java

```
3         int[][] s = {{72, 85, 90, 64}, {55, 61, 70, 80}};
4         for (int[] row : s) {  // each row is an int[]
5             int total = 0;
6             for (int x : row) {  // each element of that row
7                 total = total + x;
8             }
9             System.out.println(total);
10        }
```

The enhanced for loop on a table

```
for (int[] row : t) { for (int x : row) { ... } }   // for each row, each x in it
```

The outer variable is a whole row, an `int[]`; the inner loop is the enhanced for of week 7 on that row.

codeoutput/ForEachTable.java — Java

```
3         int[][] s = {{72, 85, 90, 64}, {55, 61, 70, 80}};
4         for (int[] row : s) {  // each row is an int[]
5             int total = 0;
6             for (int x : row) {  // each element of that row
7                 total = total + x;
8             }
9             System.out.println(total);
10        }
```

Output
codeoutput/ForEachTable.txt

```
311
266
```

Read only, and no indices: to write cells, or to know `r` and `c`, use the index loops.

Rows and columns mixed up

Q. The table has 2 rows and 3 columns. What goes wrong?

codeoutput/SwappedIndex.java — Java

```
3           int[][] t = new int[2][3];              // 2 rows, 3 columns
4           for (int r = 0; r < t[0].length; r++) {     // wrong: 3 rows?
5               for (int c = 0; c < t.length; c++) {    // wrong: 2 columns?
6                   t[r][c] = r + c;
7               }
8           }
9           System.out.println("done");
```

Rows and columns mixed up

Q. The table has 2 rows and 3 columns. What goes wrong?

codeoutput/SwappedIndex.java — Java

```
3           int[][] t = new int[2][3];               // 2 rows, 3 columns
4           for (int r = 0; r < t[0].length; r++) {     // wrong: 3 rows?
5               for (int c = 0; c < t.length; c++) {    // wrong: 2 columns?
6                   t[r][c] = r + c;
7               }
8           }
9           System.out.println("done");
```

Output — codeoutput/SwappedIndex.txt

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 2 out of bounds for length 2
        at SwappedIndex.main(SwappedIndex.java:6)
```

Rows and columns mixed up

Q. The table has 2 rows and 3 columns. What goes wrong?

codeoutput/SwappedIndex.java — Java

```
3           int[][] t = new int[2][3];              // 2 rows, 3 columns
4           for (int r = 0; r < t[0].length; r++) {     // wrong: 3 rows?
5               for (int c = 0; c < t.length; c++) {    // wrong: 2 columns?
6                   t[r][c] = r + c;
7               }
8           }
9           System.out.println("done");
```

Output — codeoutput/SwappedIndex.txt

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 2 out of bounds for length 2
        at SwappedIndex.main(SwappedIndex.java:6)
```

- The outer loop counts to 3, but there are only 2 rows: `t[2]` does not exist. The program stops with the out-of-bounds error of week 7, at line 6.
- On a square table this bug hides. Rule: `r` goes with `t.length`, `c` with `t[r].length`.

Two errors the compiler finds

Output codeoutput/CommaIndex.txt

```
CommaIndex.java:4: error: ']' expected
                t[1, 2] = 7;
                   ^
CommaIndex.java:4: error: not a statement
                t[1, 2] = 7;
                 ^
2 errors
```

`t[1, 2]` is not Java: one pair of brackets per index, `t[1][2]`.

Output codeoutput/NoRowCount.txt

```
NoRowCount.java:3: error: ']' expected
                int[][] t = new int[][3];
                                    ^
1 error
```

The number of rows is required. `new int[3][]` is allowed (the rows come later); `new int[][3]` is not.

Printing a table

codeoutput/PrintTable.java **Java**

```java
import java.util.Arrays;
public class PrintTable {
    public static void main(String[] args) {
        int[][] t = {{1, 2, 3}, {40, 50, 60}};
        System.out.println(t);                          // table
        System.out.println(Arrays.toString(t));         // rows
        System.out.println(Arrays.deepToString(t)); // cells
        for (int r = 0; r < t.length; r++) {
            for (int c = 0; c < t[r].length; c++) {
                System.out.printf("%4d", t[r][c]);   // 4 wide
            }
            System.out.println();
        }
    }
}
```

Printing a table

codeoutput/PrintTable.java — Java

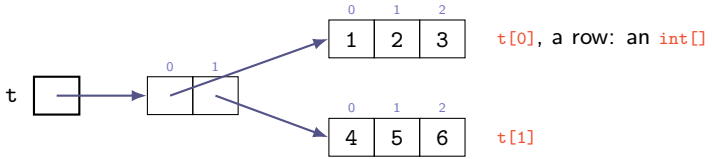
```java
import java.util.Arrays;
public class PrintTable {
    public static void main(String[] args) {
        int[][] t = {{1, 2, 3}, {40, 50, 60}};
        System.out.println(t);                          // table
        System.out.println(Arrays.toString(t));         // rows
        System.out.println(Arrays.deepToString(t)); // cells
        for (int r = 0; r < t.length; r++) {
            for (int c = 0; c < t[r].length; c++) {
                System.out.printf("%4d", t[r][c]);   // 4 wide
            }
            System.out.println();
        }
    }
}
```

Output
codeoutput/PrintTable.txt

```
[[I@2a139a55
[[I@15db9742, [I@6d06d69c]
[[1, 2, 3], [40, 50, 60]]
   1   2   3
  40  50  60
```

`println` and `Arrays.toString` show only arrows; `Arrays.deepToString` shows the cells. `printf("%4d", ...)` (week 3) makes straight columns.

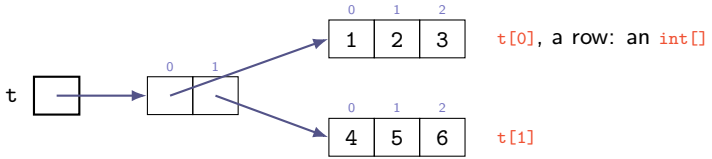
A table is an array of rows



codeoutput/RowsAreArrays.java **Java**

```java
public class RowsAreArrays {
    public static void main(String[] args) {
        int[][] t = {{1, 2, 3}, {4, 5, 6}};
        int[] row = t[1];  // a second arrow to row 1
        row[0] = 99;
        System.out.println(t[1][0]);
        t[0] = new int[] {7, 8};     // a new row 0, length 2
        System.out.println(t[0].length + " " + t[1].length);
    }
}
```

A table is an array of rows



codeoutput/RowsAreArrays.java — Java

```
1  public class RowsAreArrays {
2      public static void main(String[] args) {
3          int[][] t = {{1, 2, 3}, {4, 5, 6}};
4          int[] row = t[1];  // a second arrow to row 1
5          row[0] = 99;
6          System.out.println(t[1][0]);
7          t[0] = new int[] {7, 8};      // a new row 0, length 2
8          System.out.println(t[0].length + " " + t[1].length);
9      }
10 }
```

Output
codeoutput/RowsAreArrays.txt

```
99
2 3
```

`row` is a second arrow to row 1 (an alias, week 7). Line 7 gives the table a new, shorter row 0.

Rows of different lengths

codeoutput/Ragged.java — Java

```
1  public class Ragged {
2      public static void main(String[] args) {
3          int[][] tri = new int[4][];  // 4 rows, no row yet
4          for (int r = 0; r < tri.length; r++) {
5              tri[r] = new int[r + 1];  // row r: r + 1 cells
6              for (int c = 0; c < tri[r].length; c++) {
7                  tri[r][c] = c + 1;
8              }
9          }
10         for (int r = 0; r < tri.length; r++) {
11             for (int c = 0; c < tri[r].length; c++) {
12                 System.out.print(tri[r][c] + " ");
13             }
14             System.out.println();
15         }
16     }
17 }
```

`new int[4][]` makes 4 rows that do not exist yet; line 5 creates each one. Such a ragged table is why loops use `t[r].length`, not `t[0].length`.

Rows of different lengths

codeoutput/Ragged.java — Java

```
1  public class Ragged {
2      public static void main(String[] args) {
3          int[][] tri = new int[4][];  // 4 rows, no row yet
4          for (int r = 0; r < tri.length; r++) {
5              tri[r] = new int[r + 1];  // row r: r + 1 cells
6              for (int c = 0; c < tri[r].length; c++) {
7                  tri[r][c] = c + 1;
8              }
9          }
10         for (int r = 0; r < tri.length; r++) {
11             for (int c = 0; c < tri[r].length; c++) {
12                 System.out.print(tri[r][c] + " ");
13             }
14             System.out.println();
15         }
16     }
17 }
```

Output — codeoutput/Ragged.txt

```
1
1 2
1 2 3
1 2 3 4
```

`new int[4][]` makes 4 rows that do not exist yet; line 5 creates each one. Such a ragged table is why loops use `t[r].length`, not `t[0].length`.

On paper: draw the arrows

Draw the variables, the arrows and the rows after each line. Then answer.

```
int[][] a = {{1, 2}, {3, 4}};
int[][] b = a;
int[][] c = java.util.Arrays.copyOf(a, a.length);
c[0][0] = 9;
c[1] = new int[] {5, 6};
```

1. What is `a[0][0]`?
2. What is `a[1][0]`?
3. How many rows (`int[]`) are there?

On paper: draw the arrows

Draw the variables, the arrows and the rows after each line. Then answer.

```
int[][] a = {{1, 2}, {3, 4}};
int[][] b = a;
int[][] c = java.util.Arrays.copyOf(a, a.length);
c[0][0] = 9;
c[1] = new int[] {5, 6};
```

1. What is `a[0][0]`? 9: `c` is a new array of arrows, but to the *same* rows
2. What is `a[1][0]`? 3: `c[1]` now points to a new row; row 1 of `a` is unchanged
3. How many rows (`int[]`) are there? 3: the two rows of `a`, and `{5, 6}`

Copying a table copies only the arrows to its rows. To copy the cells, copy every row.

Common table algorithms

The patterns of week 7, by row and by column

A catalogue of table patterns

Pattern	**Question**	**Loops**	**At cell** (r, c)	**Result**
Row total	each row's sum?	r outside, c inside	$s \leftarrow s + t[r][c]$	one per row
Column total	each column's sum?	c outside, r inside	$s \leftarrow s + t[r][c]$	one per column
Largest	the largest cell?	r, c	if $t[r][c] > t[b_r][b_c]$: keep r, c	b_r, b_c
Search	where is x?	both stop early	if $t[r][c] = x$: keep r, c	r, c or -1
Count	how many match?	r, c	if match: $n \leftarrow n + 1$	n
Neighbors	the cells around	none	check the edge first	up to 4 cells
Diagonal	$t[i][i]$	one loop, i	square tables only	n cells

Every pattern of week 7 works on a table. First decide: per row, per column, or over all cells?

Row totals and column totals

codeoutput/RowColTotals.java — Java

```
3         int[][] s = {{72, 85, 90, 64},
4                      {55, 61, 70, 80},
5                      {90, 88, 93, 99}};
6         for (int r = 0; r < s.length; r++) {  // each row
7             int total = 0;
8             for (int c = 0; c < s[r].length; c++) {
9                 total = total + s[r][c];
10            }
11            System.out.println("student " + r + ": " + total);
12        }
13        for (int c = 0; c < s[0].length; c++) {  // each column
14            int total = 0;
15            for (int r = 0; r < s.length; r++) {
16                total = total + s[r][c];
17            }
18            System.out.println("quiz " + c + ": " + total);
19        }
```

Column totals: the column loop is outside, the row loop inside. `s[0].length` is the number of columns only in a rectangular table.

Row totals and column totals

codeoutput/RowColTotals.java — Java

```
3         int[][] s = {{72, 85, 90, 64},
4                      {55, 61, 70, 80},
5                      {90, 88, 93, 99}};
6         for (int r = 0; r < s.length; r++) {  // each row
7             int total = 0;
8             for (int c = 0; c < s[r].length; c++) {
9                 total = total + s[r][c];
10            }
11            System.out.println("student " + r + ": " + total);
12        }
13        for (int c = 0; c < s[0].length; c++) {  // each column
14            int total = 0;
15            for (int r = 0; r < s.length; r++) {
16                total = total + s[r][c];
17            }
18            System.out.println("quiz " + c + ": " + total);
19        }
```

Output
codeoutput/RowColTotals.txt

```
student 0: 311
student 1: 266
student 2: 370
quiz 0: 217
quiz 1: 234
quiz 2: 253
quiz 3: 243
```

Column totals: the column loop is outside, the row loop inside. `s[0].length` is the number of columns only in a rectangular table.

The largest cell

codeoutput/LargestCell.java **Java**

```
3         int[][] s = {{72, 85, 90, 64},
4                      {55, 61, 70, 80},
5                      {90, 88, 93, 99}};
6         int br = 0;  // the best so far:
7         int bc = 0;  // its row and column
8         for (int r = 0; r < s.length; r++) {
9             for (int c = 0; c < s[r].length; c++) {
10                if (s[r][c] > s[br][bc]) {
11                    br = r;
12                    bc = c;
13                }
14            }
15        }
16        System.out.println("largest " + s[br][bc]);
17        System.out.println("row " + br + ", column " + bc);
```

Keep the row *and* the column of the best so far; start with cell $(0, 0)$.

The largest cell

codeoutput/LargestCell.java — Java

```
3         int[][] s = {{72, 85, 90, 64},
4                      {55, 61, 70, 80},
5                      {90, 88, 93, 99}};
6         int br = 0;  // the best so far:
7         int bc = 0;  // its row and column
8         for (int r = 0; r < s.length; r++) {
9             for (int c = 0; c < s[r].length; c++) {
10                if (s[r][c] > s[br][bc]) {
11                    br = r;
12                    bc = c;
13                }
14            }
15        }
16        System.out.println("largest " + s[br][bc]);
17        System.out.println("row " + br + ", column " + bc);
```

Output
codeoutput/LargestCell.txt

```
largest 99
row 2, column 3
```

Keep the row *and* the column of the best so far; start with cell $(0, 0)$.

On paper: trace the largest-cell loops

`t = {{4, 9, 2}, {9, 1, 7}}`; fill one row per step, then click.

`r`	`c`	`t[r][c]`	`t[r][c] > t[br][bc]`?	`br`	`bc`
start	–	–	–	0	0
0	0	4			
0	1	9			
0	2	2			
1	0	9			
1	1	1			
1	2	7			

On paper: trace the largest-cell loops

`t = {{4, 9, 2}, {9, 1, 7}}`; fill one row per step, then click.

`r`	`c`	`t[r][c]`	`t[r][c] > t[br][bc]`?	`br`	`bc`
start	–	–	–	0	0
0	0	4	no	0	0
0	1	9	yes	0	1
0	2	2	no	0	1
1	0	9	no, equal	0	1
1	1	1	no	0	1
1	2	7	no	0	1

Cell $(0, 1)$ wins the tie: row by row, the first 9 is met first.

Searching a table

codeoutput/FindCell.java — Java

```
3         int[][] s = {{72, 85, 90, 64},
4                      {55, 61, 70, 80},
5                      {90, 88, 93, 99}};
6         int wanted = 70;
7         int fr = -1;                        // -1: not found yet
8         int fc = -1;
9         for (int r = 0; r < s.length && fr == -1; r++) {
10            for (int c = 0; c < s[r].length && fr == -1; c++) {
11                if (s[r][c] == wanted) {
12                    fr = r;
13                    fc = c;
14                }
15            }
16        }
17        System.out.println(wanted + " at " + fr + ", " + fc);
```

`fr == -1` is in both loop conditions, so both loops stop at the first match. -1 means "not found": it is never an index (week 7).

Searching a table

codeoutput/FindCell.java — Java

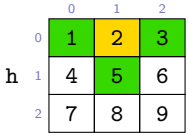
```
3         int[][] s = {{72, 85, 90, 64},
4                      {55, 61, 70, 80},
5                      {90, 88, 93, 99}};
6         int wanted = 70;
7         int fr = -1;                        // -1: not found yet
8         int fc = -1;
9         for (int r = 0; r < s.length && fr == -1; r++) {
10            for (int c = 0; c < s[r].length && fr == -1; c++) {
11                if (s[r][c] == wanted) {
12                    fr = r;
13                    fc = c;
14                }
15            }
16        }
17        System.out.println(wanted + " at " + fr + ", " + fc);
```

Output codeoutput/FindCell.txt

```
70 at 1, 2
```

`fr == -1` is in both loop conditions, so both loops stop at the first match. -1 means "not found": it is never an index (week 7).

The neighbors of a cell



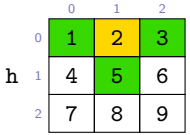
Cell $(0, 1)$ is on the top edge: it has no "up" neighbor.

codeoutput/Neighbors.java — Java

```
6         int r = 0;
7         int c = 1;  // the cell with 2
8         int sum = 0;
9         if (r > 0)                { sum = sum + h[r - 1][c]; }
10        if (r < h.length - 1)     { sum = sum + h[r + 1][c]; }
11        if (c > 0)                { sum = sum + h[r][c - 1]; }
12        if (c < h[r].length - 1) { sum = sum + h[r][c + 1]; }
13        System.out.println("cell " + h[r][c]);
14        System.out.println("sum of its neighbors " + sum);
```

Up, down, left, right: $(r-1, c)$, $(r+1, c)$, $(r, c-1)$, $(r, c+1)$. Check the edge *before* reading the cell, or the index is out of bounds.

The neighbors of a cell



Cell $(0, 1)$ is on the top edge: it has no "up" neighbor.

codeoutput/Neighbors.java — Java

```
6         int r = 0;
7         int c = 1;  // the cell with 2
8         int sum = 0;
9         if (r > 0)                { sum = sum + h[r - 1][c]; }
10        if (r < h.length - 1)     { sum = sum + h[r + 1][c]; }
11        if (c > 0)                { sum = sum + h[r][c - 1]; }
12        if (c < h[r].length - 1)  { sum = sum + h[r][c + 1]; }
13        System.out.println("cell " + h[r][c]);
14        System.out.println("sum of its neighbors " + sum);
```

Output — codeoutput/Neighbors.txt

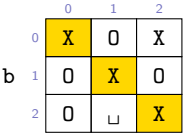
```
cell 2
sum of its neighbors 9
```

Up, down, left, right: $(r-1, c)$, $(r+1, c)$, $(r, c-1)$, $(r, c+1)$. Check the edge *before* reading the cell, or the index is out of bounds.

The diagonals of a square board

codeoutput/Diagonals.java — Java

```
3         char[][] b = {{'X', 'O', 'X'},
4                       {'O', 'X', 'O'},
5                       {'O', ' ', 'X'}};
6         int n = b.length;      // a square board
7         boolean down = true;   // b[0][0], b[1][1], b[2][2]
8         boolean up = true;     // b[0][2], b[1][1], b[2][0]
9         for (int i = 0; i < n; i++) {
10            down = down && b[i][i] == 'X';
11            up = up && b[i][n - 1 - i] == 'X';
12        }
13        System.out.println("down " + down + ", up " + up);
```



`b[i][i]` is the diagonal from the top left; `b[i][n - 1 - i]` the other one, with the mirror index of week 7. One loop is enough.

The diagonals of a square board

codeoutput/Diagonals.java — Java

```
3         char[][] b = {{'X', 'O', 'X'},
4                       {'O', 'X', 'O'},
5                       {'O', ' ', 'X'}};
6         int n = b.length;      // a square board
7         boolean down = true;   // b[0][0], b[1][1], b[2][2]
8         boolean up = true;     // b[0][2], b[1][1], b[2][0]
9         for (int i = 0; i < n; i++) {
10            down = down && b[i][i] == 'X';
11            up = up && b[i][n - 1 - i] == 'X';
12        }
13        System.out.println("down " + down + ", up " + up);
```

b

	0	1	2
0	X	O	X
1	O	X	O
2	O	␣	X

Output
codeoutput/Diagonals.txt

```
down true, up false
```

`b[i][i]` is the diagonal from the top left; `b[i][n - 1 - i]` the other one, with the mirror index of week 7. One loop is enough.

Tables and methods

A table parameter is an arrow, as in week 7

Methods that take a table

codeoutput/TableMethods.java — Java

```java
    static int[] rowTotals(int[][] t) {
        int[] totals = new int[t.length];          // one per row
        for (int r = 0; r < t.length; r++) {
            for (int c = 0; c < t[r].length; c++) {
                totals[r] = totals[r] + t[r][c];
            }
        }
        return totals;
    }
    static int[][] transpose(int[][] t) {          // t is rectangular
        int[][] result = new int[t[0].length][t.length];
        for (int r = 0; r < t.length; r++) {
            for (int c = 0; c < t[r].length; c++) {
                result[c][r] = t[r][c];
            }
        }
        return result;
    }
```

Q. Which method changes the caller's table?

Methods that take a table

codeoutput/TableMethods.java — Java

```
3     static int[] rowTotals(int[][] t) {
4         int[] totals = new int[t.length];          // one per row
5         for (int r = 0; r < t.length; r++) {
6             for (int c = 0; c < t[r].length; c++) {
7                 totals[r] = totals[r] + t[r][c];
8             }
9         }
10        return totals;
11    }
12    static int[][] transpose(int[][] t) {          // t is rectangular
13        int[][] result = new int[t[0].length][t.length];
14        for (int r = 0; r < t.length; r++) {
15            for (int c = 0; c < t[r].length; c++) {
16                result[c][r] = t[r][c];
17            }
18        }
19        return result;
20    }
```

Q. Which method changes the caller's table? Neither: both only read `t`. Writing `t[r][c]` would change the caller's table.

Calling them

codeoutput/TableMethods.java — Java

```
21      public static void main(String[] args) {
22          int[][] s = {{72, 85, 90, 64}, {55, 61, 70, 80}};
23          System.out.println(Arrays.toString(rowTotals(s)));
24          System.out.println(Arrays.deepToString(transpose(s)));
25      }
```

- `rowTotals` returns an `int[]`: one number for every row
- `transpose` returns a new `int[][]`: row `r` becomes column `r`, so a 2×4 table becomes 4×2
- Both work for any number of rows and columns; nothing is fixed to 2 or 4

Calling them

codeoutput/TableMethods.java — Java

```
21      public static void main(String[] args) {
22          int[][] s = {{72, 85, 90, 64}, {55, 61, 70, 80}};
23          System.out.println(Arrays.toString(rowTotals(s)));
24          System.out.println(Arrays.deepToString(transpose(s)));
25      }
```

Output
codeoutput/TableMethods.txt

```
[311, 266]
[[72, 55], [85, 61], [90,
    70], [64, 80]]
```

- `rowTotals` returns an `int[]`: one number for every row
- `transpose` returns a new `int[][]`: row `r` becomes column `r`, so a 2×4 table becomes 4×2
- Both work for any number of rows and columns; nothing is fixed to 2 or 4

In Eclipse: watch a table in the debugger

1. Put a breakpoint on a line inside the inner loop of `RowColTotals`
2. Run » Debug As » Java Application; the keys are the ones of week 7
3. In the Variables view, open `s`: one row per entry, `[0]`, `[1]`, `[2]`; open a row to see its cells
4. Step with F6 and watch `r`, `c` and `total`: the inner loop runs to the end before `r` changes
5. In the Expressions view, add `s[r][c]`: the current cell, at every step

Try: break inside the column-total loops and see `r` change faster than `c`.

Quick check: methods

```
static void newRow(int[][] t, int r) { t[r] = new int[t[r].length]; }
static void newTable(int[][] t) { t = new int[2][2]; }
int[][] x = {{1, 2}, {3, 4}};   newTable(x);   newRow(x, 0);
```

1. After `newTable(x)`, what is `x[0][0]`?
2. After `newRow(x, 0)`, what is `x[0][0]`?

3. Write the header of a method that returns the column totals.

Quick check: methods

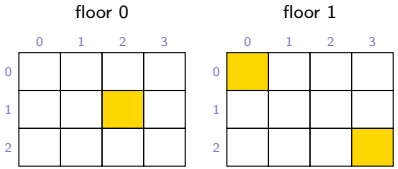
```
static void newRow(int[][] t, int r) { t[r] = new int[t[r].length]; }
static void newTable(int[][] t) { t = new int[2][2]; }
int[][] x = {{1, 2}, {3, 4}};   newTable(x);   newRow(x, 0);
```

1. After `newTable(x)`, what is `x[0][0]`? 1: only the parameter's arrow moved
2. After `newRow(x, 0)`, what is `x[0][0]`? 0: `t[r]` is a cell of the caller's array of rows, so the caller gets the new row
3. Write the header of a method that returns the column totals. `static int[] columnTotals(int[][] t)`

More dimensions

One more index, one more loop

Three dimensions: an array of tables



`taken[f][r][s]`: floor `f`, row `r`, seat `s`

`taken.length`: 2 floors

`taken[f].length`: 3 rows on floor `f`

`taken[f][r].length`: 4 seats in that row

- `new boolean[2][3][4]`: 2 tables of 3 rows of 4 seats, $2 \cdot 3 \cdot 4 = 24$ cells, all `false`
- A 3D array is an array of tables, just as a table is an array of rows

Three indices, three loops

codeoutput/ThreeD.java — Java

```java
public class ThreeD {
    public static void main(String[] args) {
        // taken[floor][row][seat]: is the seat taken?
        boolean[][][] taken = new boolean[2][3][4];
        taken[0][1][2] = true;
        taken[1][0][0] = true;
        taken[1][2][3] = true;
        int count = 0;
        for (int f = 0; f < taken.length; f++) {
            for (int r = 0; r < taken[f].length; r++) {
                for (int s = 0; s < taken[f][r].length; s++) {
                    if (taken[f][r][s]) { count++; }
                }
            }
        }
        System.out.println(count + " seats taken");
    }
}
```

Each loop takes its bound from the level above it: floors, then rows of floor `f`, then seats of that row.

Three indices, three loops

codeoutput/ThreeD.java — Java

```java
public class ThreeD {
    public static void main(String[] args) {
        // taken[floor][row][seat]: is the seat taken?
        boolean[][][] taken = new boolean[2][3][4];
        taken[0][1][2] = true;
        taken[1][0][0] = true;
        taken[1][2][3] = true;
        int count = 0;
        for (int f = 0; f < taken.length; f++) {
            for (int r = 0; r < taken[f].length; r++) {
                for (int s = 0; s < taken[f][r].length; s++) {
                    if (taken[f][r][s]) { count++; }
                }
            }
        }
        System.out.println(count + " seats taken");
    }
}
```

Output — codeoutput/ThreeD.txt

```
3 seats taken
```

Each loop takes its bound from the level above it: floors, then rows of floor `f`, then seats of that row.

N dimensions

N	Type	One element	Loops	Example
1	`int[]`	`a[i]`	1	a list of scores
2	`int[][]`	`t[r][c]`	2	students × quizzes
3	`int[][][]`	`d[f][r][s]`	3	floors × rows × seats
N	N pairs of `[]`	N indices	N nested	rare beyond 3

- Every `[]` adds one index, and usually one loop
- An N-dimensional array is an array of $(N - 1)$-dimensional arrays
- The number of cells is the product of the lengths: it grows fast

Summary

Cheat sheet

```
int[][] t = new int[3][4];                // 3 rows, 4 columns, all 0
int[][] m = {{1, 2, 3}, {4, 5, 6}};       // a list of rows
t[1][2] = 7;   int x = m[0][1];           // write, read: [row][column]
int rows = t.length;                      // the number of rows
int cols = t[0].length;                   // the length of row 0
for (int r = 0; r < t.length; r++) {              // row by row
    for (int c = 0; c < t[r].length; c++) { ... t[r][c] ... }
}
for (int[] row : t) { for (int v : row) { ... } }   // read only
int[] row1 = t[1];                        // a row is an int[]
int[][] tri = new int[4][];               // rows later: a ragged table
System.out.println(Arrays.deepToString(t));         // every cell
static int[][] transpose(int[][] t) {...}           // a table in, a table out
```

Patterns: per row, per column, over all cells; neighbors after an edge check; diagonals of square tables.

Exercises

1. Fill a 10×10 multiplication table and print it in straight columns with `printf`.
2. For the quiz table, print the average of every quiz (every column).
3. Write `static boolean isSymmetric(int[][] t)`: is `t[r][c] == t[c][r]` for every cell of a square table?
4. Count the cells that are larger than all of their (up to 4) neighbors.
5. A theater has rows of 8, 10 and 12 seats. Store it as a ragged `boolean[][]` and count the free seats.
6. For the three-dimensional `taken`, print the number of taken seats on every floor.

Where tables come back

When	Topic	Tables there
week 9	String	a `String[]` of lines is a table of characters: `lines[r].charAt(c)`
week 10	Input/Output	read a table from a file, one row per line
week 11	Exceptions, testing	today's out-of-bounds errors are exceptions; tests compare tables
week 12	Recursion	walk a grid or a maze cell by cell
CIS112	Data structures	a graph as a table: `adj[u][v]` says whether `u` and `v` are connected

Reading: bjlo2, Chapter 6, two-dimensional arrays. Decks of this topic: `arrayND-0-concepts` ... `arrayND-5-examples`.

References

The Java Tutorials, Arrays [1]; Big Java [2].

[1] The java tutorials-arrays.

[2] Cay S Horstmann.
Big Java: Early Objects.
John Wiley & Sons, 7 edition, 2019.

Questions?